

RAG End-to-End + Evaluation

Aula

Day-2 (manhã + live-coding) — 10:00 a 14:00

Disciplina

Desenvolvendo Software com IA Generativa (Mod4, 18h)

Instrutor

Nicksson Freitas — PPI/SiDi

Bem-vindo ao Day-2

- Ontem (Day-1): você fez um agente que executa ações (tool-use)
- Hoje (Day-2): você faz um agente que lê SEUS documentos (RAG)
- A diferença muda o produto: o LLM agora se ancora em fontes verificáveis que você fornece
- Resultado: respostas com citação rastreável, sem alucinação livre

Agenda do Day-2

- Aula 2.1 (10:00–11:00) — Por que RAG + embeddings + chunking
- Aula 2.2 (11:00–12:00) — Pipeline RAG end-to-end (live-coding)
- Aula 2.3 (13:00–14:00) — Evaluation com RAGAS + diagnóstico de falhas
- Tarde (14:00–16:15) — LAB-002 + LAB-003 + LAB-004
- Briefing Projeto Day-3 (16:15–16:45) + Exit-Ticket M2 (16:45–17:00)

Aula 2.1 — Por que RAG?

Recap Day-1 → Day-2

Day-1 — Tool-use	Day-2 — RAG
Pergunta: "faça algo no mundo"	Pergunta: "explique/cite algo de docs"
Solução: dar ferramentas que executam	Solução: dar contexto que ancora
Risco: ação errada	Risco: alucinação

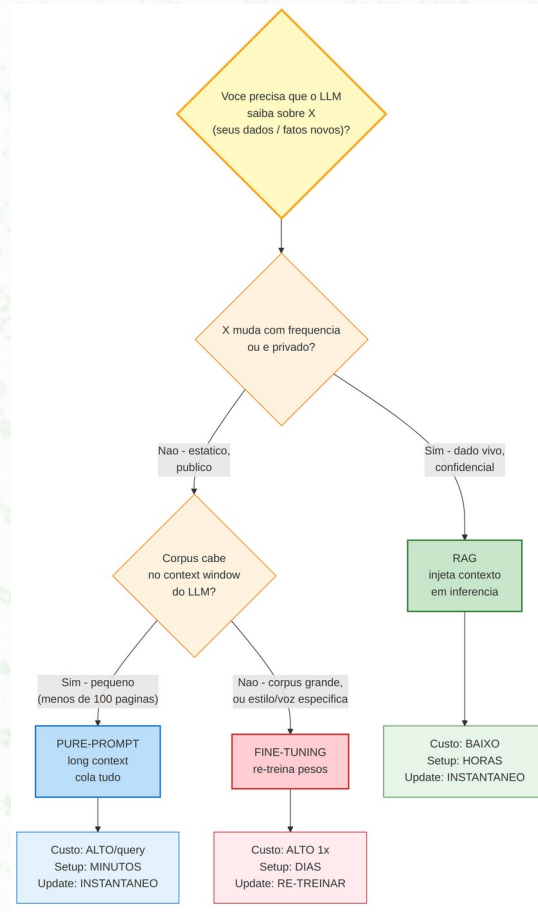
Pergunta-motivação concreta

- Cenário: 80 páginas de RFCs internos da empresa
- Pergunta no chat: "Qual é a política de retenção de logs definida no RFC-104?"
- GPT-4o não sabe: RFC-104 é doc privado, fora do training cutoff
- Pure-prompt com tudo colado: estoura context window e queima tokens
- RAG resolve: indexa os 80 PDFs 1x, e cada query recupera só 3 chunks relevantes

RAG não é "colar texto no prompt"

Abordagem	O que faz	Quando vale
Fine-tuning	Re-treina pesos com seus dados	Estilo, voz, formato fixo
RAG	Injeta contexto em inferência	Dados que mudam, privados, grandes
Pure-prompt long context	Cola tudo no prompt	Corpus pequeno, custo aceitável

Decision tree — qual abordagem usar?

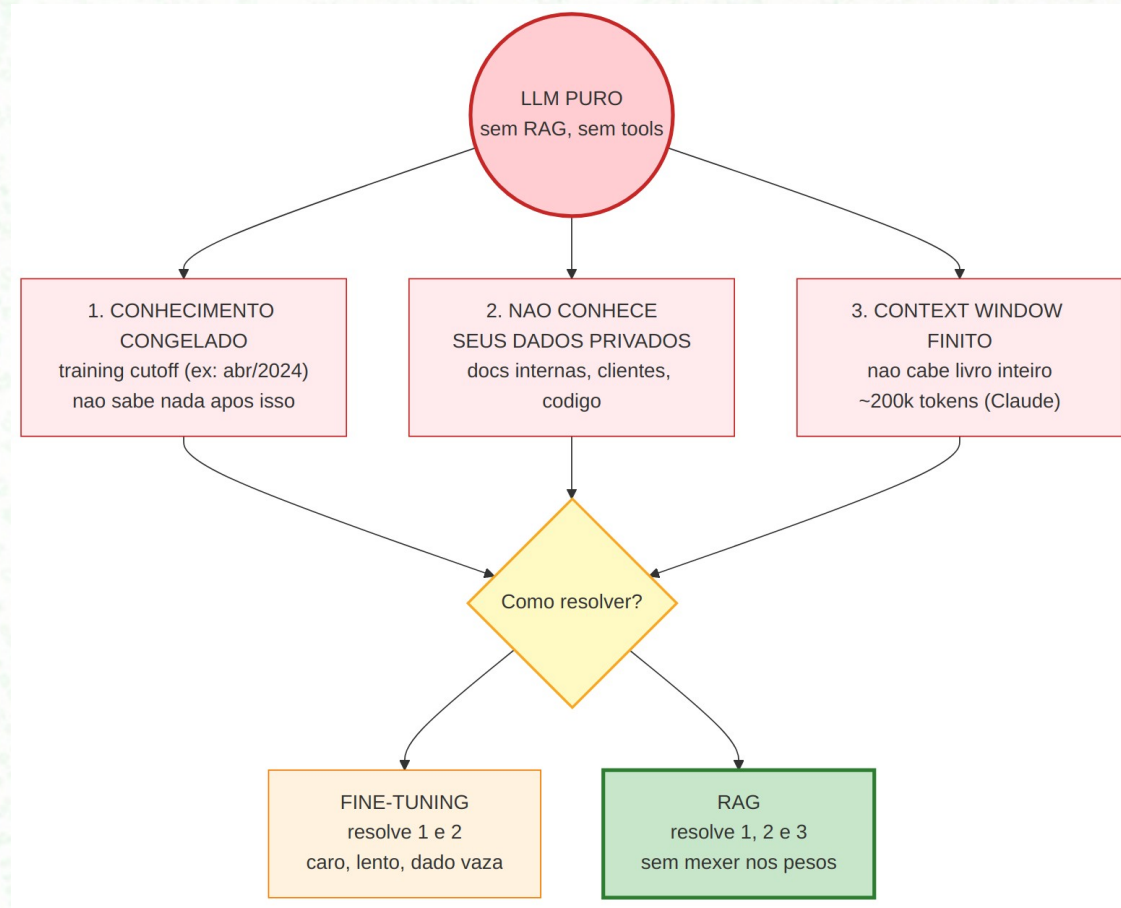


Limites do "LLM puro"

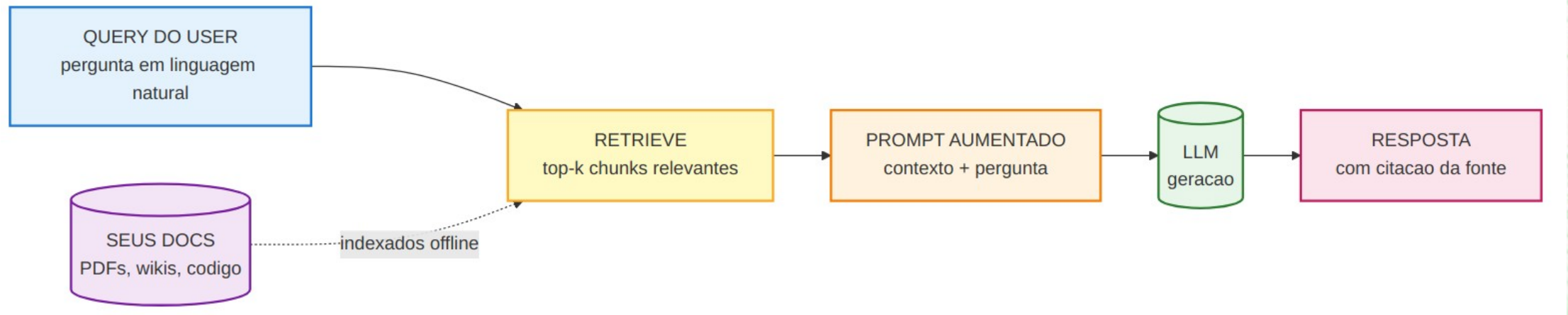
LLM tem 3 limitações inerentes:

- Conhecimento congelado no training cutoff (GPT-4o = abr/2024)
- Não conhece seus dados privados (docs internas, clientes, código)
- Context window finito — não cabe livro inteiro
- Fine-tuning resolve 1 e 2, mas é caro, lento e dado vaza no modelo
- RAG resolve os 3, sem mexer nos pesos

Limites do LLM puro — visual



RAG: ideia central



Em vez de "lembrar", o LLM "lê"

- Pipeline RAG não tenta fazer o LLM memorizar mais conteúdo
- Em inferência, anexa ao prompt os chunks recuperados do seu corpus
- O LLM gera resposta ancorada nesses trechos, com citação verificável
- Atualizar conhecimento = re-indexar docs (segundos), sem re-treinar

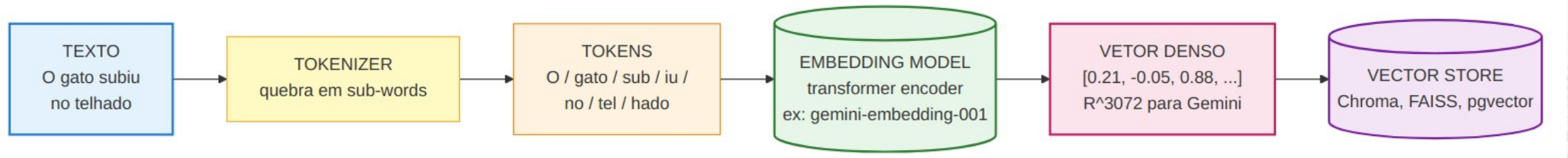
Exemplo numérico — RAG em ação

Etapa	Saída
Retrieve top-3	chunks: pg.12, pg.45, pg.61 (cos sim 0.91 / 0.87 / 0.79)
Augment	prompt = 3 chunks (~1500 tokens) + query (40 tokens)
Generate	LLM responde 80 tokens com `[RFC-104:pg.12]`

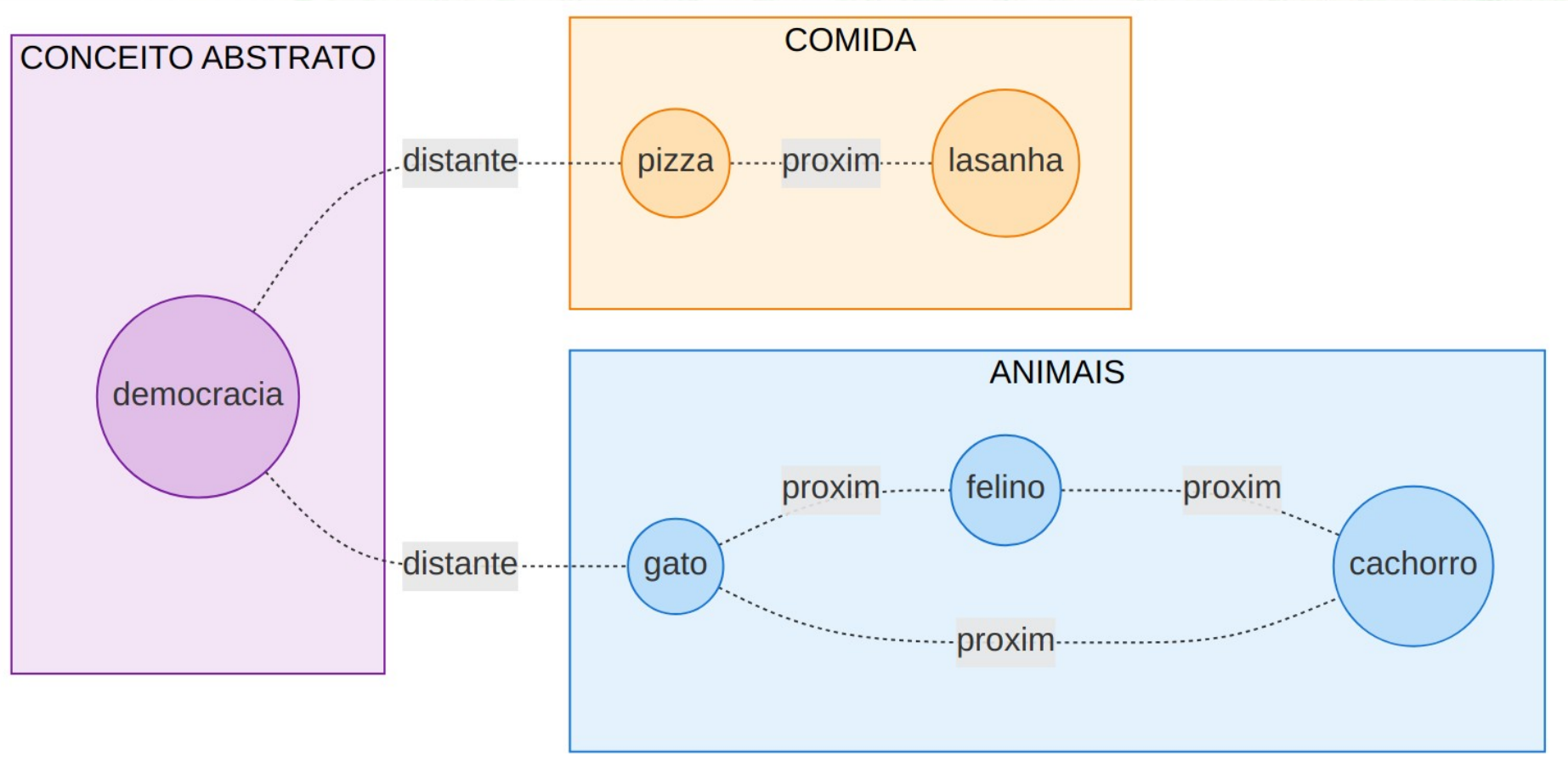
Embeddings — o que são

- Vetor denso em $\mathbb{R}^n$ ($n=384, 768, 1536, 3072\dots$) que representa significado de um texto
- Textos semanticamente similares $\rightarrow$ vetores angularmente próximos
- Exemplos (cada texto vira um vetor):
 - "O gato subiu no telhado" $\rightarrow$ `[0.21, -0.05, 0.88, ...]`
 - "O felino escalou a casa" $\rightarrow$ `[0.19, -0.03, 0.85, ...]` (similar)
 - "Pizza de calabresa" $\rightarrow$ `[-0.72, 0.41, 0.02, ...]` (distante)

Como nasce um embedding



Espaço semântico — visualização



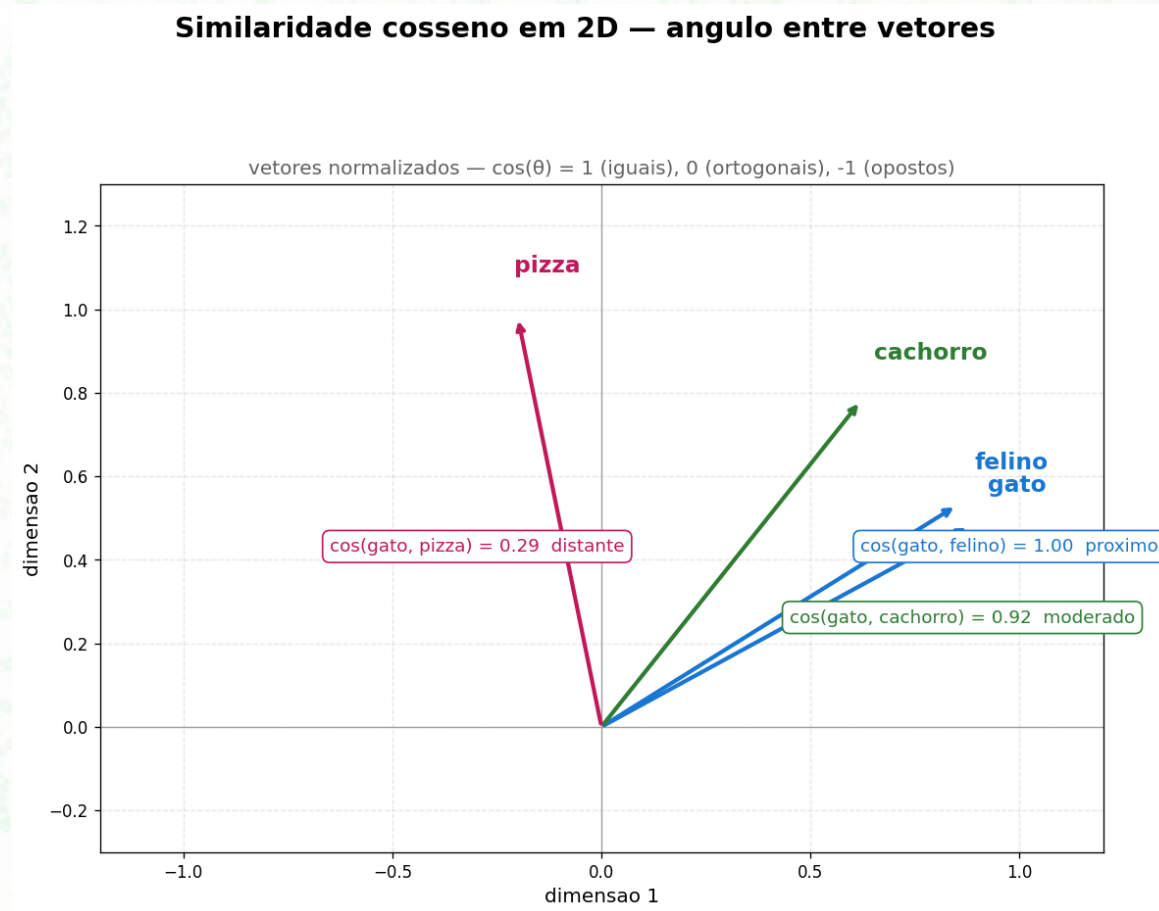
Retrieve = "qual ponto está mais perto?"

- Embedding model converte cada texto em ponto no espaço R^n
- Textos do mesmo tópico agrupam (gato, felino, cachorro)
- Tópicos distintos ficam distantes (gato vs democracia)
- Retrieve top-k = k vizinhos mais próximos da query no espaço

Similaridade cosseno

- Mede o ângulo entre vetores normalizados (não a distância)
- Fórmula: $\cos(\theta) = \frac{\text{vecu} \cdot \text{vecv}}{||\text{vecu}|| \cdot ||\text{vecv}||}$
- 1.0 → idênticos (mesma direção)
- 0.0 → ortogonais (sem relação)
- -1.0 → opostos
- Invariante à magnitude — apenas direção importa

Cosine similarity — visualização 2D



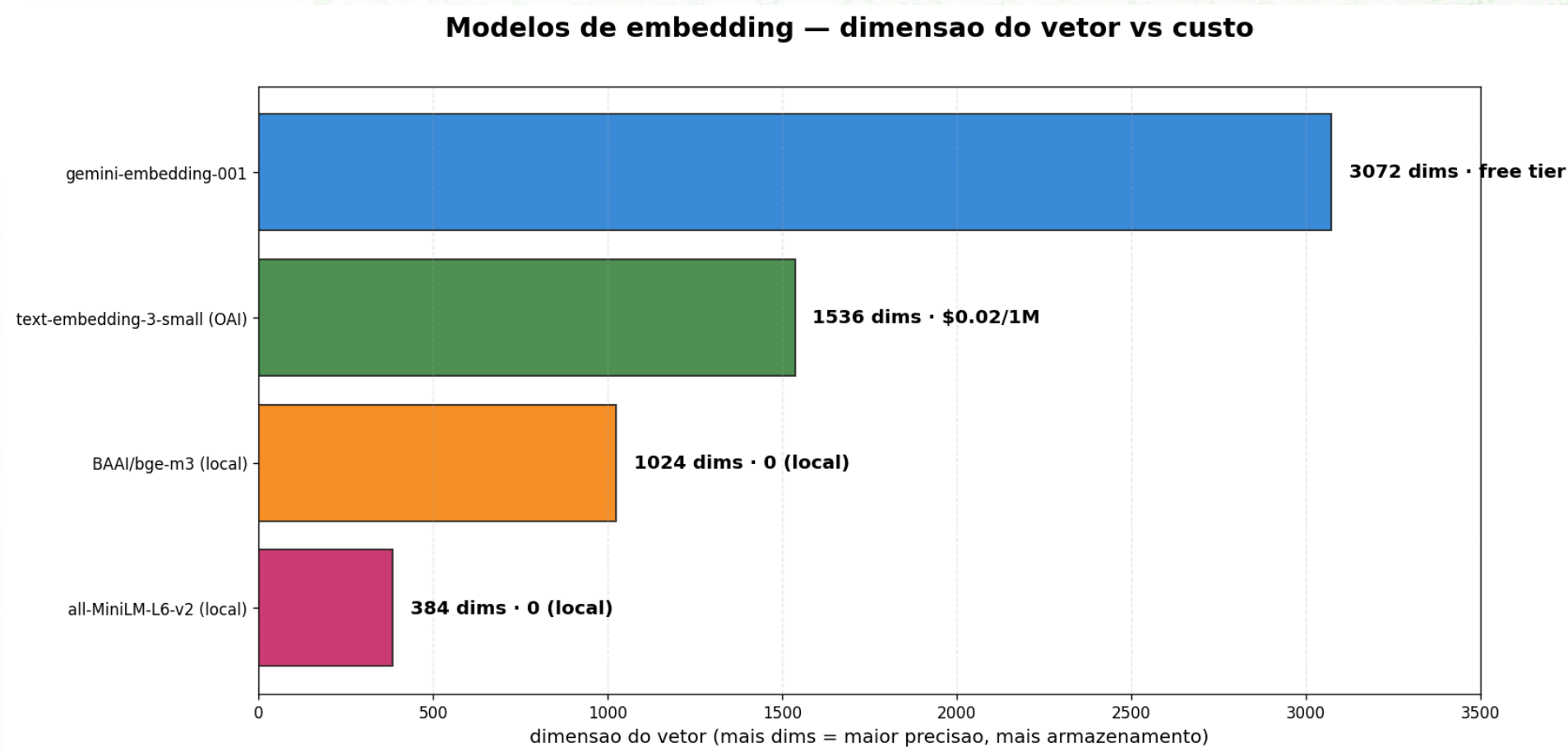
Exemplo concreto — números reais

Par	$\cos(\theta)$	Interpretação
"O gato subiu no telhado" $\times$ "O felino escalou a casa"	0.91	quase sinônimo
"O gato subiu no telhado" $\times$ "Pizza de calabresa"	0.05	sem relação
"O gato subiu no telhado" $\times$ "O cachorro pulou no sofá"	0.62	mesma família semântica

Embedding models — escolha por domínio

Modelo	Dim	Idioma	Custo	Quando usar
`gemini-embedding-001`	3072	100+	free tier	default da turma
`text-embedding-3-small` (OAI)	1536	EN-dominante	\$0.02/1M	inglês, baixo custo
`BAAI/bge-m3` (local)	1024	multi	0	sem cartão
`all-MiniLM-L6-v2` (local)	384	EN	0	dev rápido, leve

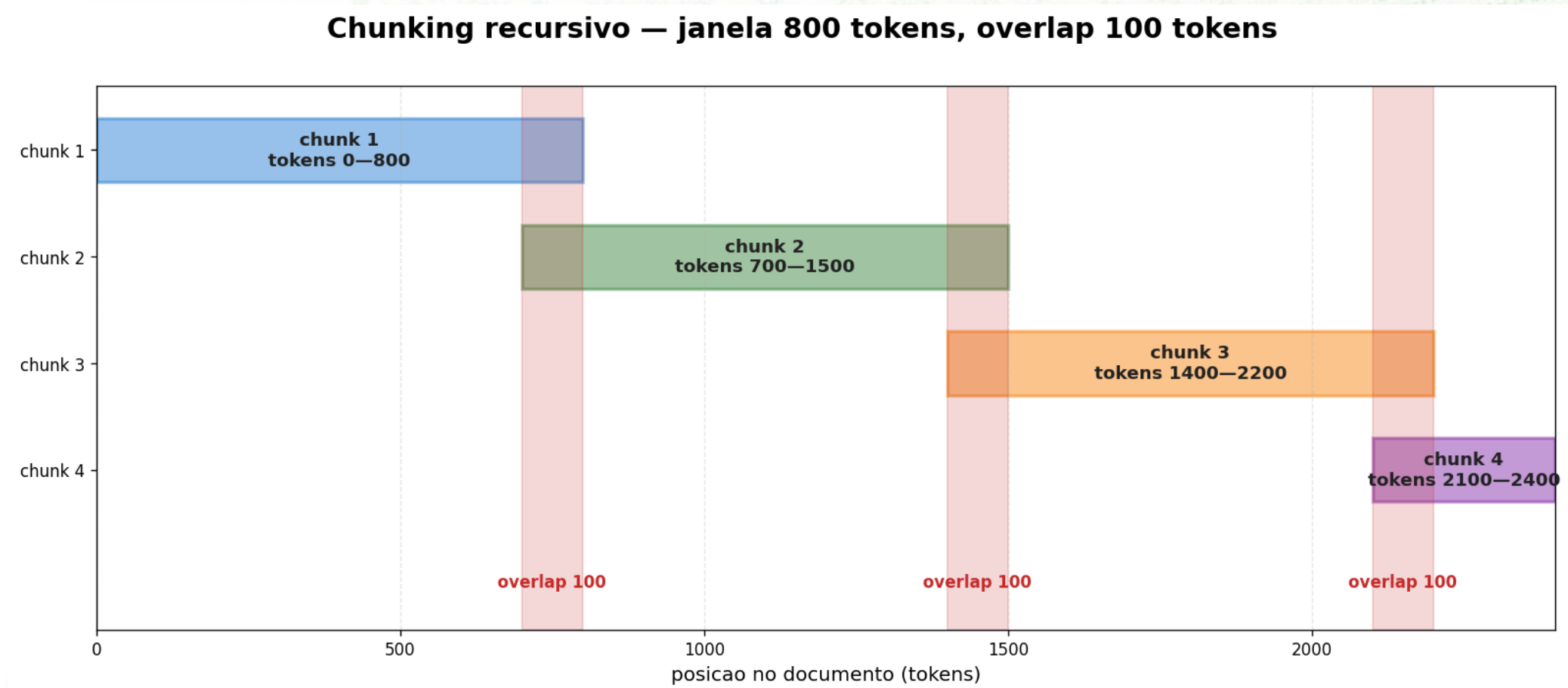
Dimensão vs custo — visualização



Chunking — por que importa

- Documentos não cabem inteiros no embedder — quebrar em chunks preserva contexto local
- Livro inteiro num vetor → perde granularidade (não acha trecho específico)
- Frase por frase → perde contexto (vetor curto, sem vizinhança)
- A escolha do `chunk\_size` afeta retrieval E generation — decisão crítica do pipeline

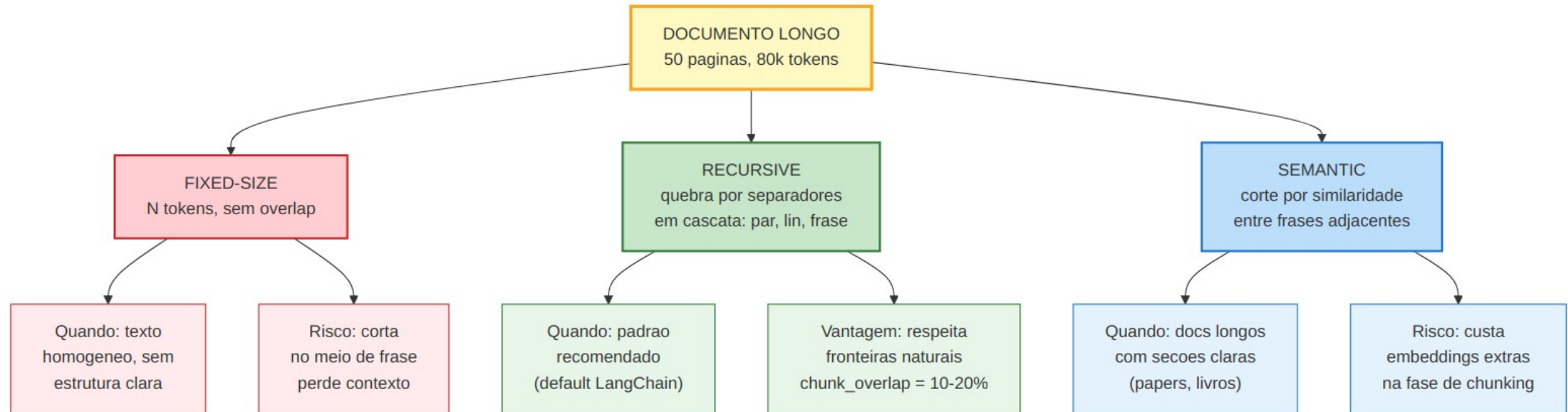
Janela de chunking — overlap em ação



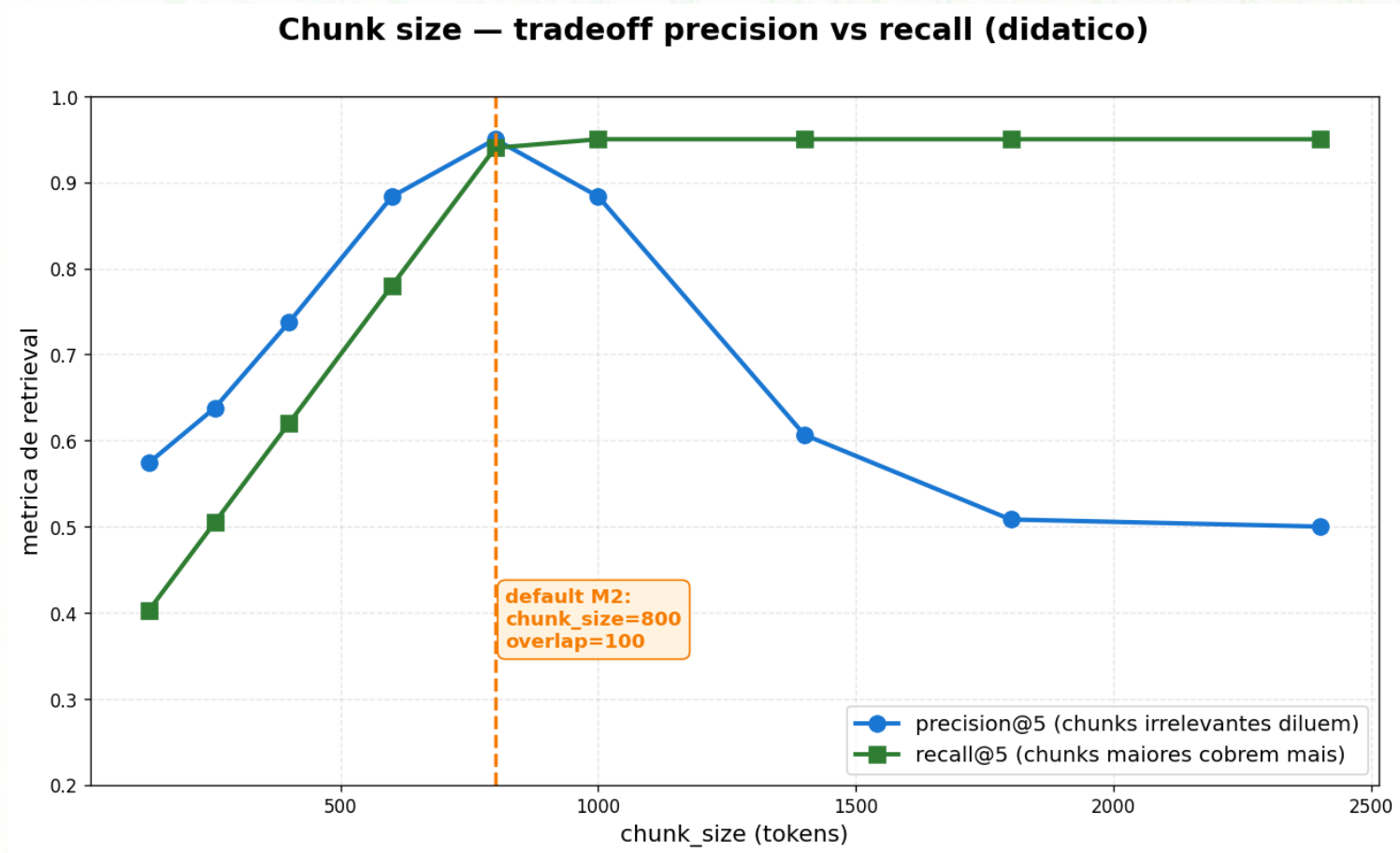
Chunking strategies

Estratégia	Como	Quando
Fixed-size	N tokens, sem overlap	Texto homogêneo
Recursive	Quebra por separadores em cascata (`\n\n`, `\n`, `.`)	Padrão recomendado
Semantic	Corte por similaridade entre frases	Documentos longos com seções claras

Chunking — comparação visual



Chunk size tradeoff — precision vs recall



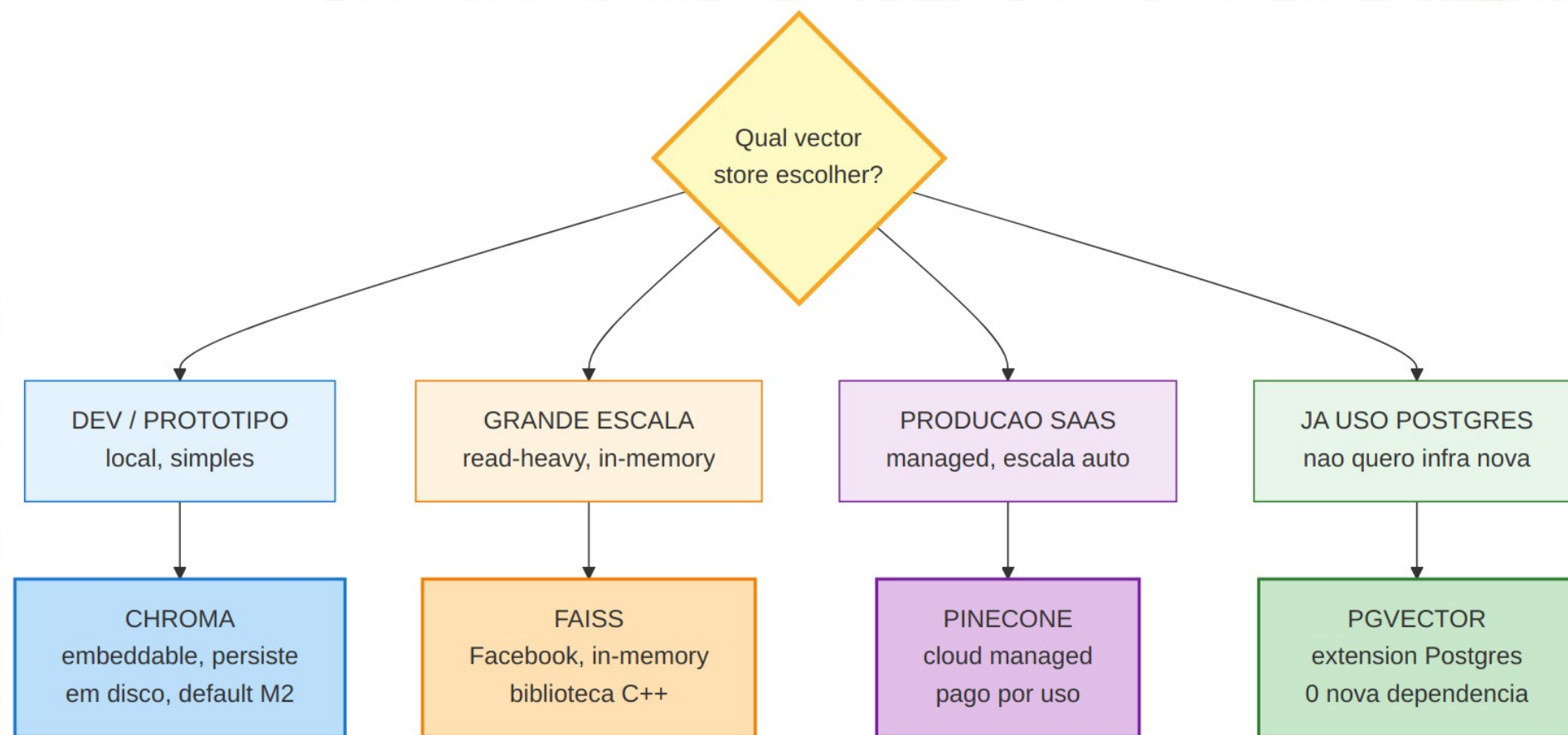
Default da turma: 800 / 100

- Chunks muito pequenos (<256 tokens): precision OK, mas recall despenca — perde contexto
- Chunks muito grandes (>1400): captura tudo, mas chunks irrelevantes diluem o sinal
- Sweet spot: `chunk\_size=800`, `overlap=100` — ponto onde as duas curvas se cruzam saudáveis
- Regra prática: 500–1000 tokens cobre 90% dos casos didáticos

Vector stores

Vector store	Características	Quando usar
Chroma	Embeddable, persistência em disco, fácil	Dev e protótipos
FAISS	In-memory, máxima performance	Grande escala read-heavy
Pinecone	Managed cloud, escala automática	Produção SaaS
pgvector	Postgres extension	Você já usa Postgres

Vector stores — decision tree



Recap Aula 2.1 → transição para 2.2

Você agora tem os blocos LEGO do RAG:

- Embeddings transformam texto em vetor
- Cosine mede proximidade semântica
- Chunking quebra docs em pedaços indexáveis
- Vector store guarda os vetores

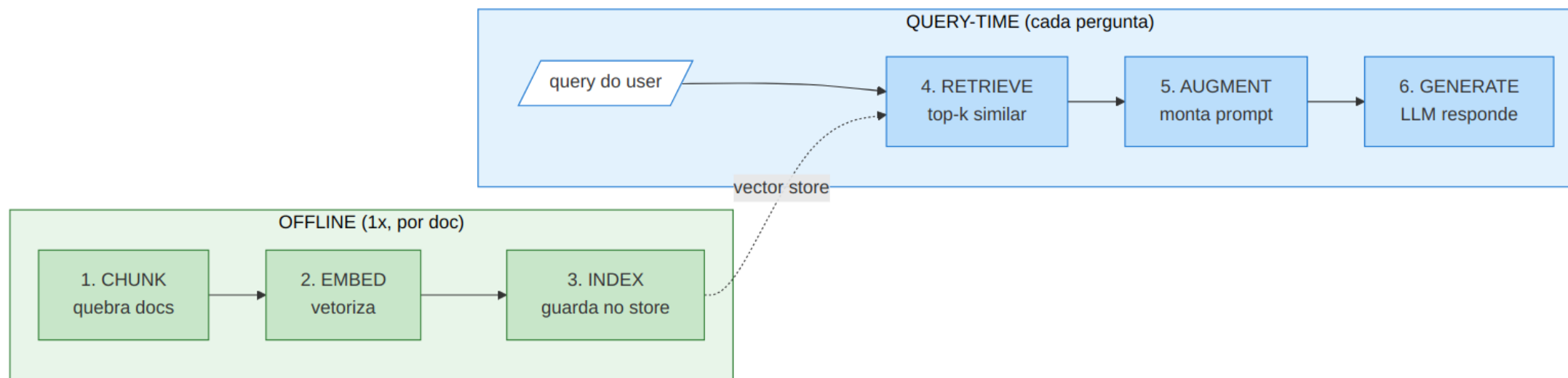
Próximo passo: encaixar tudo num pipeline executável.

Aula 2.2 — Pipeline RAG E2E

Pipeline RAG — montando as 6 etapas

- Os conceitos do Aula 2.1 combinam em 6 etapas distintas
- Cada etapa pode ser otimizada independentemente — `chunk_size`, `embedding model`, `top_k`, `prompt`
- Saber em qual etapa um problema vive = debug rápido vs trocar tudo de uma vez
- Mental model crítico antes de ver código

As 6 etapas — visão integrada



Offline vs query-time — separação crítica

Quando roda	Etapas	Frequência	Otimização
Offline (1× por doc)	chunk → embed → index	Setup inicial + re-ingestion	Lote, paralelizar
Query-time (por pergunta)	retrieve → augment → generate	A cada chat turn	Latência, cache

Prompt template — 4 elementos críticos

- Âncora ao contexto: "Responda APENAS com base no contexto abaixo"
- Fallback explícito: "Se não estiver no contexto, diga 'Não encontrado'"
- Instrução de citação: "Sempre cite a fonte usando `[arquivo:pagina]`"
- `temperature=0` no client → resposta determinística para mesmo contexto

Prompt template — código

```
```python
RAG_PROMPT = """Voce e um assistente tecnico. Responda APENAS
com base no contexto abaixo. Se a informacao nao estiver no
contexto, diga "Nao encontrado no corpus". Sempre cite a fonte
usando o formato [arquivo:pagina].
CONTEXTO:
{context}
PERGUNTA: {question}
RESPOSTA:"""
```
```


Anatomia do prompt aumentado

- Query original: "Qual a política de retenção de logs?"
- Retrieve top-3 traz chunks dos PDFs com page tags
`[arquivo:pagina]`
- Augment monta o prompt injetando contexto + pergunta
- Generate produz resposta citando as páginas específicas
- Veja o prompt completo no próximo slide

Anatomia — prompt real enviado ao LLM

'''

CONTEXTO:

[RFC-104:pg.12] Logs de acesso retidos por 90 dias em S3 cold storage...

[RFC-104:pg.45] Logs de aplicação seguem janela de 30 dias antes...

[RFC-104:pg.61] Auditoria mantém cópia indefinida em bucket WORM...

PERGUNTA: Qual a politica de retencao de logs?

RESPOSTA:

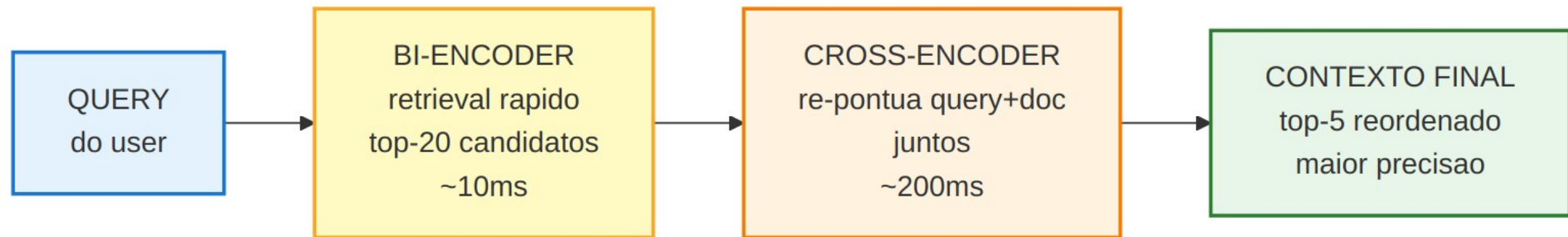
'''

- LLM responde ancorado: "Acesso 90d, app 30d, auditoria indefinida [RFC-104:pg.12/45/61]"

Re-ranking — quando faz sentido

- Bi-encoder (embedding) é rápido (~10ms) mas pode trazer falsos positivos no top-5
- Cross-encoder pontua query+doc juntos e reordena os candidatos
- Trade-off: +precisão, ~200ms de latência/query, +custo se for via API
- Vale a pena quando: corpus técnico, queries ambíguas, precision crítica

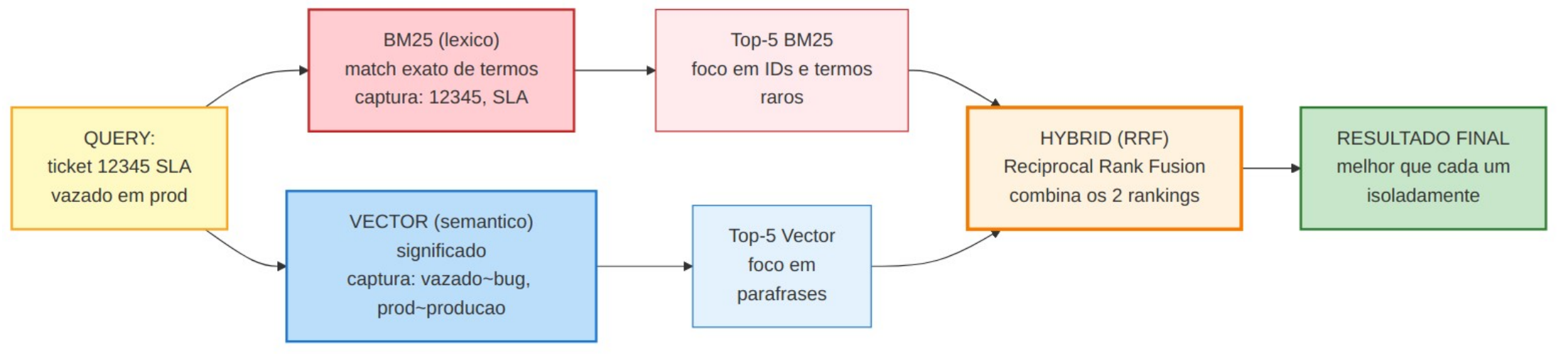
Re-ranking — fluxo bi-encoder → cross-encoder



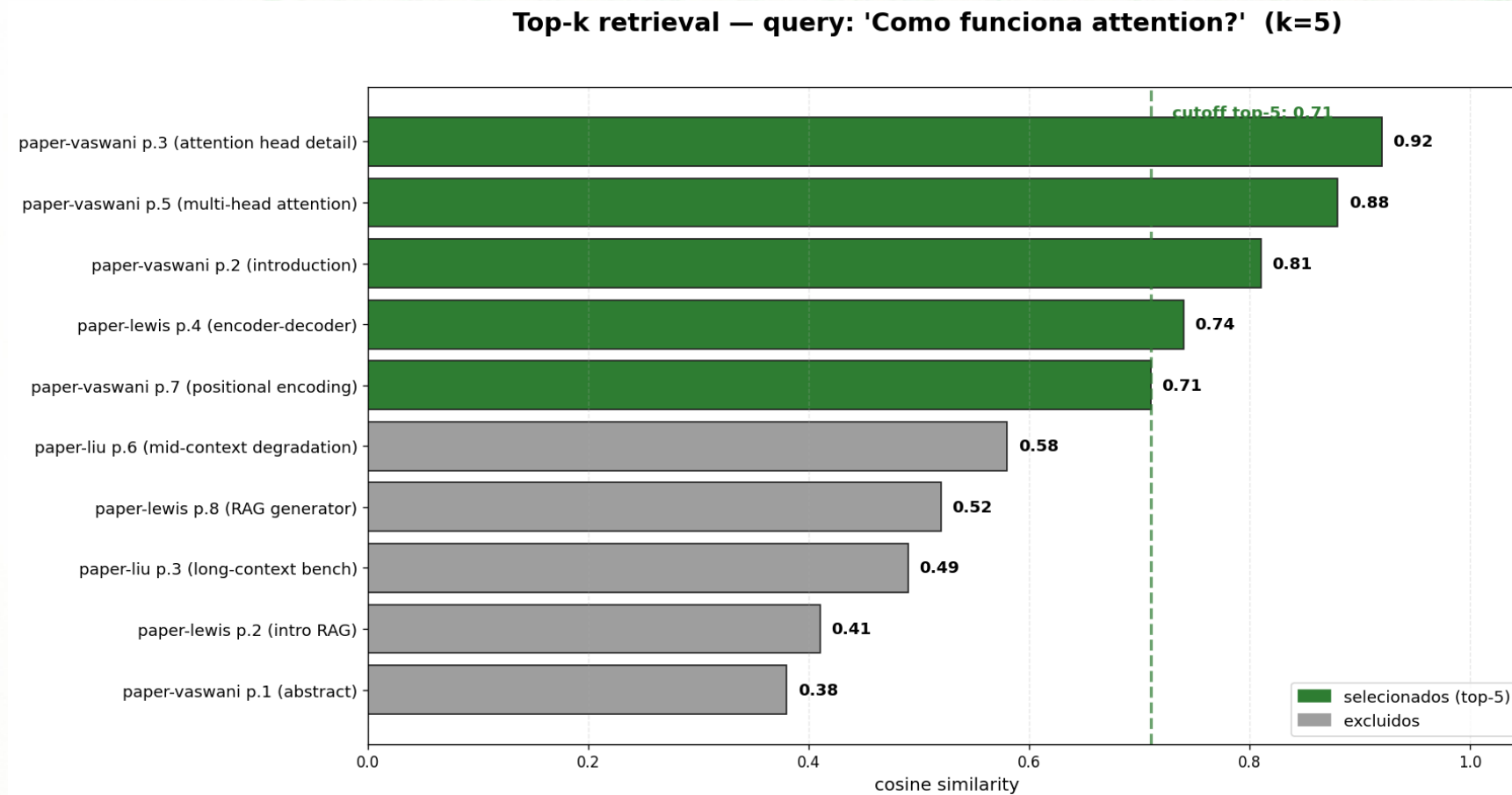
Hybrid retrieval (BM25 + vector)

| Approach | Captura | Falha em |
|--------------|---|-----------------------|
| BM25 | Match exato de termos (IDs, nomes, códigos) | Paráfrases |
| Vector | Significado semântico | Termos exatos raros |
| Hybrid (RRF) | Ambos | Pouco; combina forças |

Hybrid — quando léxico + semântico vencem isoladamente



Top-k em ação — scores reais



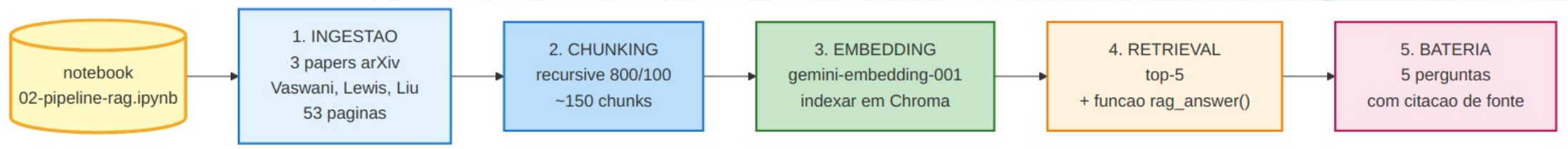
Como ler o gráfico de top-k

- Eixo X: cosine similarity entre query e chunk (0—1)
- Cutoff em 0.71: corta no top-5; abaixo entra ruído de outros papers
- Top-5 captura 5 trechos do paper certo (Vaswani — attention)
- Adicionar o 6º (Liu/Lewis) diluiria o contexto com paper diferente
- Regra de bolso: começar com top-5; ajustar com base no RAGAS (próxima aula)

Live-coding (13:00–14:00) — o que faremos

- Abrir ``notebooks/02-pipeline-rag.ipynb`` lado a lado
- Construir as 5 etapas em sequência (cada uma em ~10 min)
- Vamos parar em cada etapa para inspecionar saída intermediária
- No final: bateria de 5 perguntas com citação de fonte funcionando

Live-coding — sequência visual



Recap Aula 2.2 → transição para 2.3

- Você já tem um pipeline funcional: query entra, resposta sai com citação
- Mas funciona BEM? Como saber se:
 - O `chunk\_size` escolhido é ótimo?
 - O top-5 captura o que precisa?
 - A resposta gerada está ancorada nos chunks (ou alucinou)?
- Resposta: métricas automatizadas — tema da Aula 2.3

Aula 2.3 — Evaluation com RAGAS

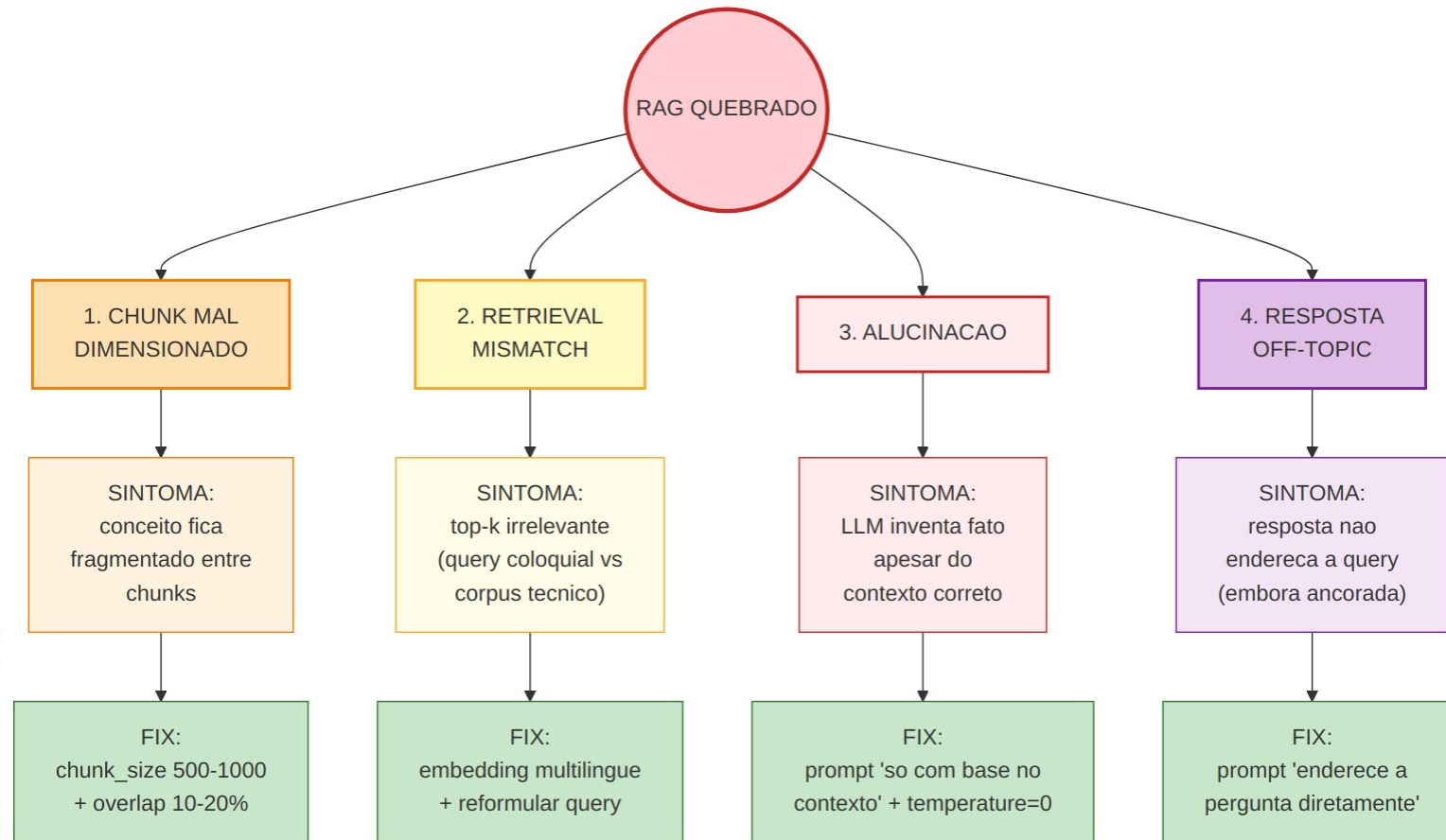
Sem métricas, RAG vira fé

- "Funcionou no meu teste manual" não escala
- Pipeline pode estar quebrado em 4 lugares diferentes simultaneamente
- Métricas automatizadas respondem perguntas como:
 - Mudei o chunk_size: melhorou ou piorou?
 - O embedding novo é melhor que o velho?
 - O prompt está alucinando ou só está vago?
- Sem números, todo ajuste é palpite

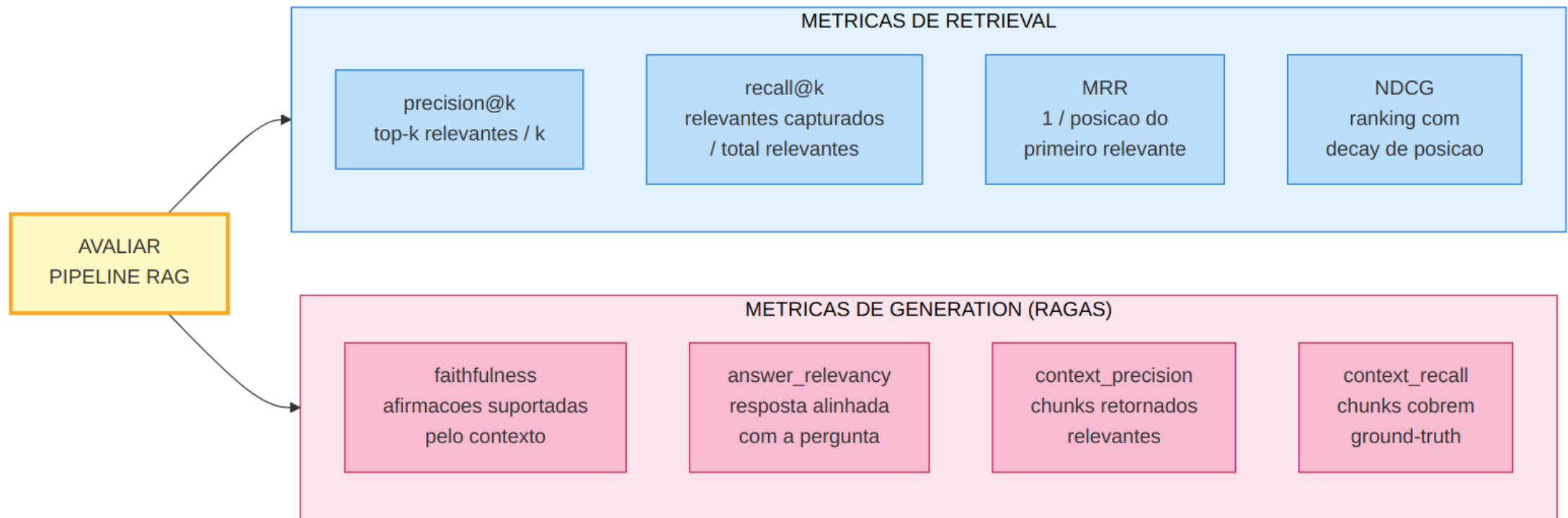
RAG pode quebrar de 4 formas

- Chunk mal dimensionado → fragmentação de conceitos
- Retrieval mismatch → top-k irrelevante (query coloquial vs corpus técnico)
- Alucinação → LLM inventa apesar do contexto correto
- Resposta off-topic → não responde a query, embora ancorado
- Cada falha tem sintoma diferente nas métricas — é isso que vamos diagnosticar

4 modos de falha — sintoma → fix



Métricas — retrieval vs generation



Métricas de retrieval

| Métrica | O que mede |
|-------------|--|
| precision@k | top-k relevantes / k |
| recall@k | relevantes capturados / total relevantes |
| MRR | 1 / posição do primeiro relevante |
| NDCG | Ranking metric com decay de posição |

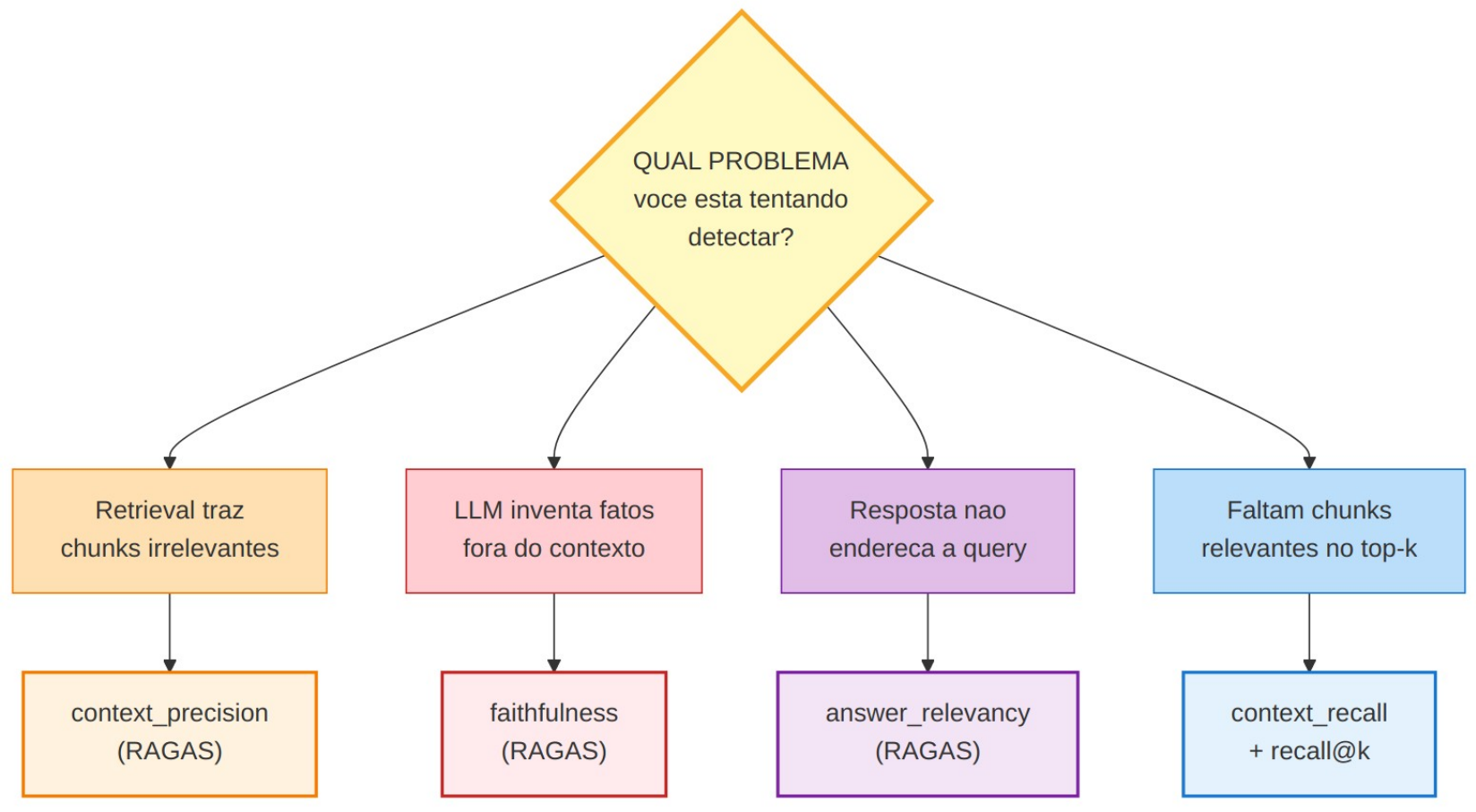
Métricas de generation (RAGAS)

| Métrica | O que mede | Detecta |
|-------------------|-------------------------------------|----------------|
| faithfulness | Afirmações suportadas pelo contexto | Alucinação |
| answer_relevancy | Resposta alinhada com a pergunta | Off-topic |
| context_precision | Chunks retornados relevantes | Retrieval ruim |
| context_recall | Chunks cobrem ground-truth | Recall ruim |

Faithfulness na prática — exemplo

| Resposta gerada | Faithfulness | Por quê |
|---------------------------------|--------------|----------------------------------|
| "8 cabeças paralelas" | 1.00 | Cada afirmação ancorada |
| "8 cabeças e 12 layers" | 0.50 | "12 layers" não está no contexto |
| "16 cabeças com self-attention" | 0.00 | Inventou número e detalhe |

Qual métrica usar? — decision tree



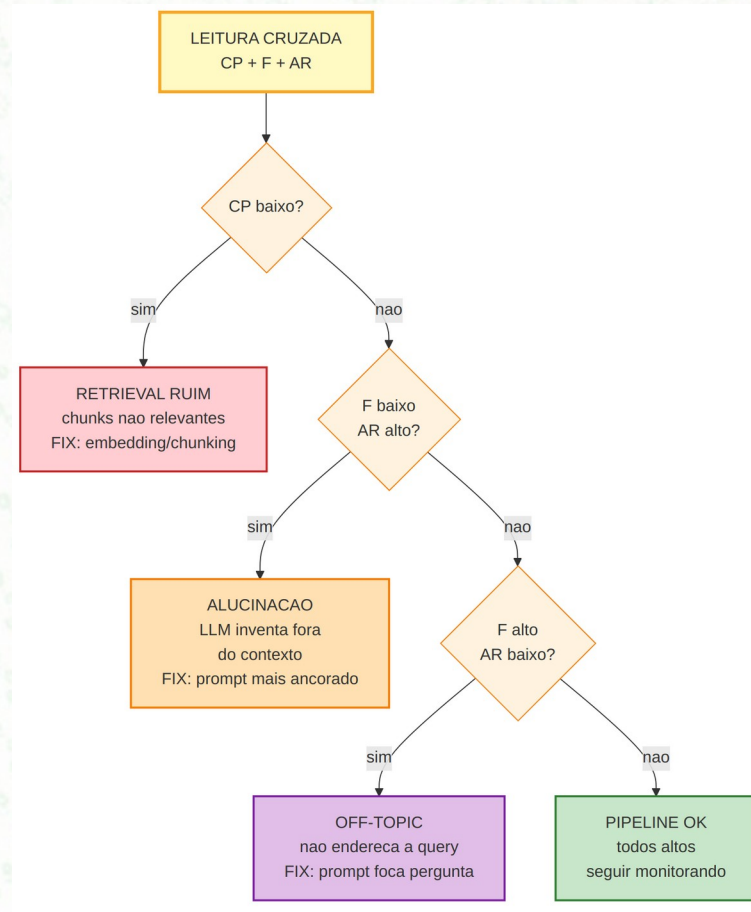
Por que decision tree e não "rode tudo"?

- Rodar todas as 4 métricas RAGAS custa ~\$0.05/query (LLM-as-judge)
- Em 100 queries de eval → \$5 por experimento; multiplica rápido em iteração
- Saber qual métrica responde qual pergunta acelera diagnóstico em 5×
- Padrão profissional: começa pelo sintoma → escolhe a métrica certa → 1 experimento

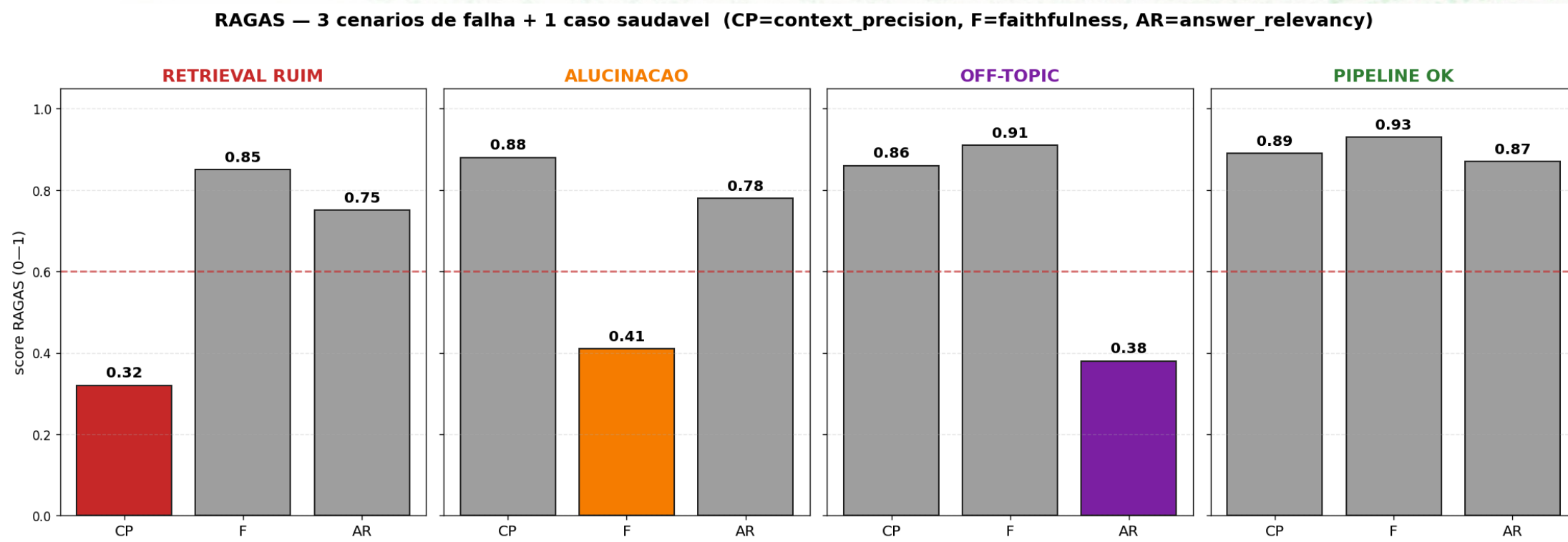
Diagnóstico por padrão de métricas

| CP | F | AR | Diagnóstico |
|-------|-------|-------|-------------------------------------|
| baixo | — | — | Retrieval ruim → embedding/chunking |
| alto | baixo | alto | Alucinação → prompt mais ancorado |
| alto | alto | baixo | Off-topic → prompt foca na pergunta |
| alto | alto | alto | OK ✓ |

Diagnóstico cruzado — fluxograma



3 cenários reais — RAGAS visualizado



Cada falha tem assinatura única

- Retrieval ruim: CP baixo, F e AR normais (LLM gerou bem, mas com contexto errado)
- Alucinação: CP alto, F baixo, AR alto (contexto certo, mas LLM inventou)
- Off-topic: CP alto, F alto, AR baixo (tudo correto, mas resposta não endereça)
- Pipeline OK: os 3 acima do cutoff de 0.6 — seguir monitorando
- Treine 5 segundos olhando o gráfico → diagnóstico imediato

Erro comum: "RAGAS deu 0.9, então tá bom?"

| Limitação | Impacto |
|-------------------------------|---|
| Judge model também alucina | Score pode ser otimista demais |
| PT-BR vs EN-dominante | Judges em EN penalizam PT-BR sutilmente |
| Resposta evasiva ganha pontos | "Não sei" às vezes scora alto em faithfulness |
| Dataset pequeno (10 Q/A) | Variância alta entre runs |

Live-coding RAGAS (continua na tarde)

Abrimos ``notebooks/03-ragas-evaluation.ipynb``:

- Dataset de eval: 10 Q/A com ground-truth manual
- Rodar pipeline sobre cada query, coletar contexts + answers
- Calcular faithfulness + answer_relevancy + context_precision
- Inspeccionar o pior caso (menor faithfulness)
- Gerar CSV + gráfico de barras

Tarde — 3 Labs + Briefing + Exit- Ticket

Tarde (14:00–16:15) — 3 labs encadeados

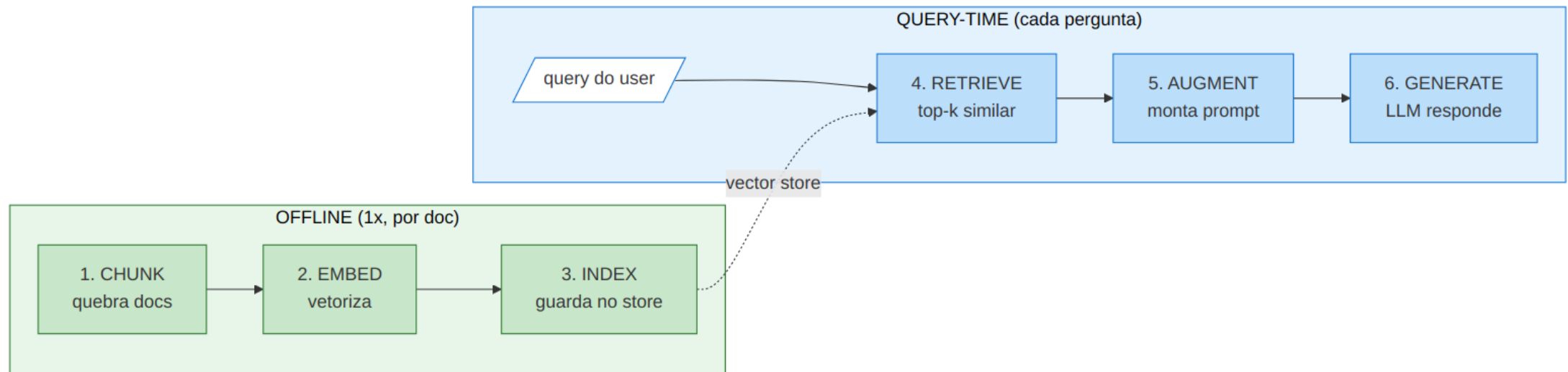
| Lab | Tempo | Objetivo |
|---------|--------|--|
| LAB-002 | 90 min | Pipeline RAG end-to-end (notebook 02) |
| LAB-003 | 60 min | RAGAS sobre LAB-002 (notebook 03) |
| LAB-004 | 50 min | Diagnóstico de 3 cenários de falha (notebook 04) |

Exit-Ticket M2 (16:45–17:00)

5 questões cobrindo M2-O1..M2-O5:

- Ordem do pipeline RAG (MCQ)
- Debug: ``collection.count()=250`` em 500 chunks (short)
- Métrica que detecta alucinação (MCQ)
- Diagnóstico $CP=0.85$, $F=0.92$, $AR=0.45$ (análise)
- RAG sobre 2000 e-mails curtos PT-BR (justify)

Recap visual do Day-2



Tudo que você viu hoje, em 2 linhas

- Fluxo (6 etapas): chunk → embed → index → retrieve → augment → generate
- Avaliação (3 métricas): context_precision + faithfulness + answer_relevancy
- Com isso você diagnostica e melhora um pipeline RAG em ≤ 5 minutos
- Próximo passo (Day-3): tirar do notebook e colocar em produção

Próximo encontro: 2026-06-08 — Day-3

M3: Projeto Portfólio em Produção

Passa de "funciona no notebook" para "deployado, com custo medido, em portfólio público".

Pré-requisito para Day-3:

- Definir mentalmente: qual problema você quer resolver?
- Ter um corpus em mente (livro, docs, código, transcripts)
- Decidir se vai Tier 1 (Gemini Free), Tier 2 (pago com \$5 free) ou Tier 3 (Ollama local)

Briefing Projeto Day-3 (16:15– 16:45)

O que você vai construir

- Projeto em duplas (2 alunos), entrega 17:00 do Day-3
- Integra LLM + RAG + Tool-use já vistos em M1/M2
- Roda em URL pública (Streamlit Cloud é default)
- Demo gravada em vídeo ≤ 3 min
- Vale 60% da nota da disciplina (ASMT-006)

Modalidades — escolha no kickoff Day-3

| Modalidade | Como | Para quem |
|------------------------------|--|--|
| A. Template + corpus próprio | Clona template, troca corpus, ajusta tools | Default para a turma — caminho mais seguro |
| B. Projeto autoral | Liberdade total de arquitetura | Quem já tem ideia + repo próprio rodando |

Rubrica em 4 critérios

| Critério | Peso | Banda "sólido" exige |
|----------|------|--------------------------------------|
| Técnica | 40% | e2e funciona + erros tratados + logs |
| README | 30% | problem + diagrama + métricas |
| Custo | 20% | mede custo + cache com hit-rate |
| Demo | 10% | URL acessível + sem crash |

5 ideias pré-validadas (Modalidade A)

- 5 problemas com corpus + tool + scope já validados em ``delivery/projetos/PROBLEM-IDEAS.md``:
 - Chat sobre changelog de framework Python
 - Q&A em compliance regulatório (LGPD)
 - Suporte de produto via base de tickets
 - Assistente de arXiv para subdomínio
 - Recap de transcripts de reunião
- Cada ideia vem com corpus indicado e tool de exemplo
- Você pode pegar uma e personalizar, ou propor a sua

Como chegar pronto ao Day-3

Antes de 2026-06-08, faça:

- ☐ Forme a dupla (até 2026-06-04 via Forms)
- ☐ Escolha modalidade A ou B
- ☐ Defina problema + corpus (1 frase cada)
- ☐ Leia `projeto-portfolio-anexo-tecnico.pdf` (cache, routing, observabilidade, deploy)
- ☐ Verifique API key funcionando
- No Day-3 você chega para construir, não para decidir

Obrigado!

Dúvidas + plantão até 17:30.

Lab notebooks executados ficam como seu "rascunho" para M3 — vamos reaproveitar a função ``rag_answer()`` no projeto integrador (não jogue fora).